CIÊNCIA DE DADOS TEÓRICA E APLICADA

Fundamentos de Modelos Não-Lineares

Um guia didático e matematicamente rigoroso do ponto de vista do leigo: da rigidez da reta à elegância probabilística das curvas sigmoides e métodos de Newton.

1. Do Exemplo do Apartamento ao Limite da Reta

No início de seus estudos em Ciência de Dados, o modelo linear básico assume que o mundo funciona como uma proporção contínua e infinita: **Preço = a × Área + b**. Se um imóvel de 40 m² custa R$ 90.000 e um de 100 m² custa R$ 150.000, calculamos analiticamente uma inclinação constante (R$ 1.000 por metro quadrado) e um valor base fixo (R$ 50.000). Isso funciona bem para grandezas proporcionais sem barreiras.

A Premissa do Modelo Linear

O modelo assume que cada acréscimo na variável de entrada (**x**) gera um impacto proporcional direto na saída (**y**). O efeito de passar de 40 para 50 m² é exatamente igual ao de passar de 500 para 510 m². Sua fórmula é direta:

$$y = w_0 + w_1x_1 + \ldots + w_Dx_D = Xw$$

Onde a Reta Fracassa no Mundo Real?

O mundo real possui **saturação, limites físicos e decisões binárias**. Uma pessoa não pode estar 140% infartada ou -20% grávida. Probabilidades existem estritamente entre 0 e 1 (0% a 100%). Uma reta comum extrapolaria para números absurdos como −5 ou +18.

$$y_{previsto} \notin [-\infty, +\infty] \rightarrow \text{Exige Limites } [0, 1]$$

2. Por que Precisamos de Modelos Não-Lineares?

Dizemos que um modelo é **não-linear** quando as taxas de variação não são constantes ou quando a variável de resposta não responde como uma mera soma de fatores multiplicados por pesos fixos.

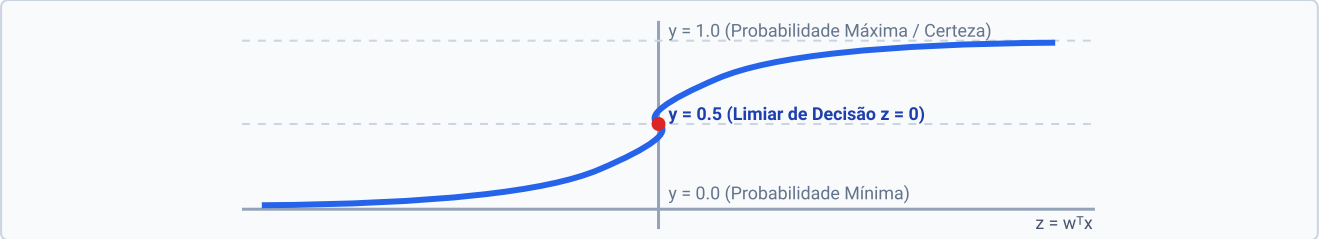
Analogia Intuitiva para Leigos: A Iluminação de um Quarto

Se você está em um quarto completamente escuro e acende 1 vela, a diferença de luminosidade é colossal (grande salto). Se você já tem 100 velas acesas e acende a vela 101, seus olhos mal notam a mudança (efeito de saturação). A reta assume que a 101ª vela ilumina tanto quanto a primeira; o modelo não-linear entende o rendimento decrescente.

Critério de Comparação	Modelo Linear (Ex: Regressão Linear)	Modelo Não-Linear (Ex: Regressão Logística)
Espaço de Saída (y)	Qualquer número real contínuo (−∞ a +∞)	Limitado a um intervalo [0, 1] ou regiões curvas
Natureza do Problema	Previsão de quantidades (Preço, Temperatura)	Probabilidades, Decisões Binárias (Sim/Não, 0/1)
Solução Matemática	Solução analítica exata via Equação Normal	Otimização numérica iterativa (Gradiente / Newton)
Sensibilidade	Mesma inclinação constante do início ao fim	Alta sensibilidade no centro, saturação nos extremos

3. O Elo de Ligação: A Função Sigmoide σ(z)

Para transformar uma combinação linear ilimitada $\mathbf{z = w^T x}$ em uma probabilidade real contida estritamente entre 0 e 1, a Ciência de Dados utiliza uma das funções não-lineares mais famosas da matemática: a **Função Logística (ou Sigmoide)**.



Fórmula Canônica da Sigmoide: $\sigma(z) = 1 / (1 + e^{-z}) \equiv e^{z} / (1 + e^{z})$

Propriedades Notáveis

- Se $\mathbf{z \to +\infty}$, $e^{-z} \to 0$, logo $\sigma(z) \to \mathbf{1}$.
- Se $\mathbf{z \to -\infty}$, $e^{-z} \to +\infty$, logo $\sigma(z) \to \mathbf{0}$.
- Se $\mathbf{z = 0}$, $e^{0} = 1$, logo $\sigma(0) = 1 / (1 + 1) = \mathbf{0.5}$.
- **Simetria Perfeita:** $1 - \sigma(z) = \sigma(-z)$.

Interpretação Probabilística

Em problemas de classificação binária (onde o resultado real $y \in \{0, 1\}$), definimos a probabilidade de ocorrência do evento como:

$$Pr(Y = 1 \mid x, w) = \sigma(w^T x) = p$$
$$Pr(Y = 0 \mid x, w) = 1 - \sigma(w^T x) = 1 - p$$

4. O Truque da Derivada da Sigmoide (Passo a Passo)

Para treinar o modelo, precisaremos calcular derivadas. O caderno de anotações destaca a elegância com que a derivada da sigmoide se resolve em função de si mesma utilizando a **Regra da Cadeia**:

Dedução Completa da Derivada dσ(x) / dx:

Reescrevemos como potência: $\sigma(x) = (1 + e^{-x})^{-1}$.

1. Derivando a camada externa pela regra do tombo: $-1 \times (1 + e^{-x})^{-2}$.
2. Multiplicando pela derivada interna do expoente: $-e^{-x}$.
3. Multiplicação dos sinais negativos: $(-1) \times (-e^{-x}) = +e^{-x}$.
4. Agrupando em frações: $[1 / (1 + e^{-x})] \times [e^{-x} / (1 + e^{-x})]$.
5. Como $e^{-x} / (1 + e^{-x}) = 1 - \sigma(x)$, concluímos:

$$\mathbf{d\sigma(x) / dx = \sigma(x) \times [1 - \sigma(x)]}$$

5. Por que não usar o Erro Quadrático Médio (MSE)?

No modelo linear, minimizávamos o Erro Quadrático Médio $E(w) = (1/N) \sum (y - \hat{y})^2$ porque a função de erro formava uma "tigela redonda" perfeita (um paraboloide convexo), possuindo apenas um ponto mínimo global. Porém, quando inserimos a função não-linear sigmoide dentro do quadrado, **a curva da função de erro fica cheia de ondulações, buracos e mínimos locais**. O algoritmo de descida do gradiente ficaria preso em poços falsos.

6. A Distribuição de Bernoulli e a Verossimilhança

Para dados binários (0 ou 1, Cara ou Coroa, Doente ou Saudável), a probabilidade de uma única observação $\mathbf{y_i}$ ocorrer segue a **Distribuição de Bernoulli**:

$$Pr(Y_i = y_i \mid x_i, w) = [p_i]^{y_i} \times [1 - p_i]^{1 - y_i} \qquad \text{onde} \quad p_i = \sigma(w^T x_i)$$

Como essa fórmula funciona na prática?

- Se o paciente estava **realmente doente (y = 1)**: $[p]^1 \times [1 - p]^0 = \mathbf{p}$ (queremos que p seja o mais próximo de 1.0 possível).
- Se o paciente estava **saudável (y = 0)**: $[p]^0 \times [1 - p]^1 = \mathbf{1 - p}$ (queremos que 1 − p seja o mais próximo de 1.0 possível).

A Verossimilhança Global (Likelihood)

A probabilidade conjunta de toda a nossa base de dados (assumindo amostras independentes) é a multiplicação das probabilidades de cada pessoa:

$$L(w) = \prod_{i=1}^{N} [\ \sigma(w^T x_i)\ ]^{y_i} \times [\ 1 - \sigma(w^T x_i)\ ]^{1 - y_i}$$

7. O Truque Computacional do Logaritmo (Log-Likelihood)

Multiplicar milhares de probabilidades pequenas (como 0.001 × 0.004...) faz o computador travar por *underflow numérico* (o número fica tão minúsculo que vira zero absoluto no hardware). A solução da Ciência de Dados é aplicar o **logaritmo natural (ln)**, pois o logaritmo transforma multiplicações em somas:

Construção da Função de Erro E(w) (Entropia Cruzada Binária)

Como algoritmos de otimização em Machine Learning são construídos para **minimizar erro** (descer até o fundo do vale) e não para maximizar ganho, definimos o Erro como o negativo da Log-Verossimilhança:

$$E(w) = - \ln L(w) = - \sum_{i=1}^{N} \{\ y_i \ln[\sigma(w^T x_i)] + (1 - y_i) \ln[1 - \sigma(w^T x_i)]\ \}$$

Substituindo algebricamente σ e log-odds: $E(w) = \sum_{i=1}^{N} \{\ \ln[1 + e^{w^T x_i}] - y_i (w^T x_i)\ \}$

8. O Cálculo do Gradiente $\nabla E(w)$ via Regra da Cadeia

Para saber para onde atualizar os pesos **w**, precisamos da derivada da função de erro em relação a cada peso. Esse vetor de derivadas parciais é o **Gradiente**. Ele atua como um "radar de inclinação" indicando a direção de subida mais íngreme. Como queremos diminuir o erro, caminhamos no sentido oposto ($-\nabla E$).

Dedução Rigorosa Termo a Termo

Dada a função: $E(w) = \sum_{i=1}^{N} \{ \ln[1 + e^{w^T x_i}] - y_i w^T x_i \}$.

1. A derivada do segundo termo é imediata: $\partial(-y_i w^T x_i) / \partial w = -y_i x_i$.
2. Para o termo com logaritmo, aplicamos a regra da cadeia: $d \ln(u) / dw = (1/u) \times (du/dw)$:
   - $u = 1 + e^{w^T x_i} \rightarrow du/dw = e^{w^T x_i} \times x_i$.
   - Resultado: $[e^{w^T x_i} / (1 + e^{w^T x_i})] \times x_i = \sigma(w^T x_i) \times x_i$.
3. Juntando ambos os termos e colocando x_i em evidência:

$$\nabla E(w) = \sum_{i=1}^{N} x_i \times [\sigma(w^T x_i) - y_i]$$

A Beleza da Conexão Linear vs. Não-Linear

Observe a simetria impressionante com a regressão linear! Em ambos os modelos, o gradiente tem a estrutura exata: **Entrada × (Previsão − Realidade)**. A única diferença matemática é que no modelo linear a previsão é $\hat{y} = w^T x$, enquanto no modelo não-linear a previsão é passada pela curva suave da sigmoide $p = \sigma(w^T x)$.

9. Representação Matricial Elegante: $\nabla E(w) = X^T(p - y)$

Em programação para Ciência de Dados (como em bibliotecas NumPy ou PyTorch), calcular esse somatório linha por linha através de loops convencionais é computacionalmente inviável. Agrupamos todas as operações em matrizes:

Estrutura dos Vetores e Matrizes

- $\mathbf{X^T}$: Matriz transposta com dimensão (D × N).
- **p**: Vetor coluna de previsões probabilísticas $[\sigma(w^T x_1), ..., \sigma(w^T x_N)]^T$.
- **y**: Vetor coluna de gabaritos reais $[y_1, ..., y_N]^T$.
- **(p − y)**: Vetor coluna de erros individuais de tamanho (N × 1).

Interpretação Física

A subtração $(p - y)$ mede o erro instantâneo de cada exemplo. Ao multiplicar pela transposta X^T, o erro é distribuído e ponderado entre cada uma das características (features), ajustando especificamente os pesos das variáveis que mais falharam.

10. A Matriz Hessiana (H): Medindo a Curvatura da Superfície

Se a primeira derivada (o Gradiente) nos informa a **inclinação do terreno**, a segunda derivada (a **Matriz Hessiana**) nos informa a **curvatura** — ou seja, se o vale onde estamos descendo tem formato de uma tigela larga e suave ou de um cânion estreito e pontiagudo.

Diferença Vital entre Modelos Lineares e Não-Lineares

- **No Modelo Linear:** A 2ª derivada é $H = (2/N)\ X^T X$. Ela é **constante**! O formato da tigela nunca muda, independente de onde os pesos w estejam.
- **No Modelo Não-Linear:** A 2ª derivada **depende dinamicamente de w**! A curvatura varia a cada passo, pois cada ponto da curva sigmoide tem uma taxa de mudança diferente.

$$H = \partial^2 E(w) / \partial w \partial w^T = \partial \nabla E(w) / \partial w = \sum_{i=1}^{N} x_i \left[\partial \sigma(w^T x_i) / \partial w \right] = \sum_{i=1}^{N} x_i \times p_i(1 - p_i) \times x_i^T$$

11. A Forma Matricial da Hessiana: $H = X^T P X$

No formato matricial compacto, a Matriz Hessiana é obtida através de uma matriz de ponderação diagonal **P**, onde os elementos fora da diagonal são todos zeros e a diagonal principal contém as variâncias de Bernoulli de cada amostra:

Matriz Diagonal P (Ponderações)

$$P = \text{diag}\left[p_1(1 - p_1),\ p_2(1 - p_2),\ \ldots,\ p_N(1 - p_N) \right]$$

Note que $p(1 - p)$ atinge valor máximo de 0.25 quando $p = 0.5$ (máxima incerteza) e vai a zero nos extremos (certeza absoluta).

Dimensão e Propriedades de H

- Dimensão: **(D × D)**, onde D é a quantidade de atributos.
- É uma matriz **simétrica** ($H = H^T$).
- Como $p_i(1 - p_i) \geq 0$ para todo i, H é *semidefinida positiva*, garantindo que a função de custo do modelo logístico é estritamente convexa (sem mínimos locais!).

12. O Método de Newton-Raphson vs. Gradiente Descendente

Com a Hessiana, podemos usar o **Método de Newton-Raphson** (também conhecido como Mínimos Quadrados Reponderados Iterativamente - IRLS):

Algoritmo	Equação de Atualização dos Pesos	Comportamento Prático
Gradiente Descendente	$w^{[t+1]} = w^{[t]} - \eta \nabla E(w)$	Dá passos cegos baseados apenas na inclinação; pode ziguezaguear e levar milhares de passos.
Método de Newton	$w^{[t+1]} = w^{[t]} - H^{-1} \nabla E(w)$	Usa a curvatura (H) para saltar diretamente rumo ao fundo; converge com extrema precisão em 5 a 10 iterações.

13. Exemplo Prático Calculado na Ponta do Lápis

Para fixar o aprendizado em nível leigo, vamos acompanhar uma iteração completa de atualização de pesos com apenas 2 pacientes e 1 característica:

Dados do Problema: Prever Risco Cardíaco (0 = Saudável, 1 = Risco Alto)

- **Paciente 1:** $x_1 = [1, 2]$ (bias = 1, colesterol = 2). Diagnóstico Real: $\mathbf{y_1 = 1}$.
- **Paciente 2:** $x_2 = [1, -1]$ (bias = 1, colesterol = −1). Diagnóstico Real: $\mathbf{y_2 = 0}$.
- **Pesos Iniciais:** $w^{[0]} = [0, 0]^T$. Taxa de Aprendizado: $\eta = 0.5$.

Resolução Passo a Passo:

1. **Combinação Linear (z):**
   - $z_1 = 0{\times}1 + 0{\times}2 = 0 \rightarrow p_1 = \sigma(0) = \mathbf{0.5}$
   - $z_2 = 0{\times}1 + 0{\times}(-1) = 0 \rightarrow p_2 = \sigma(0) = \mathbf{0.5}$
2. **Erros Individuais (p − y):**
   - $e_1 = p_1 - y_1 = 0.5 - 1 = \mathbf{-0.5}$ (subestimou o paciente doente)
   - $e_2 = p_2 - y_2 = 0.5 - 0 = \mathbf{+0.5}$ (superestimou o paciente saudável)
3. **Cálculo do Gradiente ($\sum x_i \times e_i$):**
   - $\nabla E(w) = [1, 2]^T \times (-0.5) + [1, -1]^T \times (+0.5) = [-0.5, -1.0]^T + [0.5, -0.5]^T = \mathbf{[0.0, -1.5]^T}$
4. **Atualização do Peso $w^{[1]} = w^{[0]} - \eta \nabla E$:**
   - $w_0 = 0 - 0.5 \times (0.0) = \mathbf{0.0}$
   - $w_{colesterol} = 0 - 0.5 \times (-1.5) = \mathbf{+0.75} \rightarrow$ O modelo aprendeu que maior colesterol aumenta o risco!

14. A Família Completa dos Modelos Não-Lineares em ML

A regressão logística é a porta de entrada não-linear. O ecossistema de Ciência de Dados expande essa base em diferentes direções para capturar fronteiras ainda mais complexas:

Árvores & Florestas

Dividem o espaço com cortes retos perpendiculares (if/else), modelando descontinuidades bruscas que nenhuma fórmula matemática contínua consegue expressar.

SVMs com Kernel

Projetam os dados para um espaço de dimensão superior (ex: de 2D para 3D) onde dados antes inseparáveis tornam-se linearmente separáveis por um hiperplano.

Redes Neurais Artificiais

Empilham dezenas de camadas com matrizes de pesos seguidas de funções de ativação não-lineares (como Sigmoide ou ReLU), funcionando como aproximadores universais de funções.

Síntese para Revisão de Provas e Entrevistas

Modelos lineares assumem relações aditivas sem limites. Modelos não-lineares mapeiam a realidade através de funções de ligação (sigmoide), resolvem dados probabilísticos via Máxima Verossimilhança (Log-Likelihood) e navegam pela função de erro com vetores de gradiente guiados pela curvatura da Matriz Hessiana.